DS605 - Fundamentals of Machine Learning

Lecture 05-06 - Measures of Proximity & Invariance to Transformation

Lecture Notes

Prepared by: **Arpit Rana**

Overview

These notes accompany the Lecture 05-06 slide deck, which continues directly from Lecture 04. Having established what is typical and how spread out a single attribute is, this lecture asks the natural next question: given two ShopCart customers - or two products, two documents, two time series - how alike are they? The honest answer is that there is no single correct notion of "alike"; there are many measures of proximity, each suited to a different kind of data, and choosing the wrong one is among the most common mistakes in applied recommendation systems, clustering, and anomaly detection. Part 3 builds a toolbox of proximity measures - Minkowski distances, the metric axioms, binary coefficients, cosine similarity, Mahalanobis distance - and a decision tree for choosing among them. Part 4 introduces correlation and mutual information as measures of the relationship between attributes and closes with the idea of invariance: does a proximity measure yield the same answer if you change units, currency, or scale?

LEARNING OBJECTIVES

By the end of this lecture you should be able to: (1) distinguish similarity from dissimilarity and convert between them; (2) compute Minkowski distances for r = 1, 2 and infinity, and check whether a proximity measure satisfies the metric axioms; (3) compute the Simple Matching Coefficient, the Jaccard coefficient, and cosine similarity, and know which one fits sparse binary or non-negative data; (4) explain why Euclidean distance can fail on correlated or unequally scaled attributes, and how Mahalanobis distance fixes this; (5) compute Pearson correlation and mutual information, and explain why mutual information catches non-linear relationships that correlation misses; (6) determine whether a given proximity measure is invariant to a change of scale, offset, or both; and (7) combine mixed quantitative and qualitative attributes into a single proximity measure using Gower's coefficient.

Part 3 - Measures of Proximity

Suppose ShopCart wants to build a "customers like you also bought..." recommendation. Should similarity be measured with Euclidean distance? Cosine similarity? Correlation? Jaccard? Every one of these is correct mathematics, and every one gives a different answer. Picking the wrong one is the single most common mistake in applied recommendation systems, clustering, and anomaly detection - and resolving that opening question is the goal of Part 3.

3.1 Vocabulary: similarity and dissimilarity

Together, similarity and dissimilarity are called proximity - a numerical measure of how alike (or unalike) two data objects are. Dissimilarity is the degree to which two objects differ, usually expressed in the range [0,1] or sometimes [0, infinity). Similarity is the degree to which two objects are alike, usually non-negative and often in [0,1], with 0 meaning completely unalike and 1 meaning identical. The proximity between two objects is built up from the proximity between their corresponding individual attributes.

3.2 Where proximity measures are used

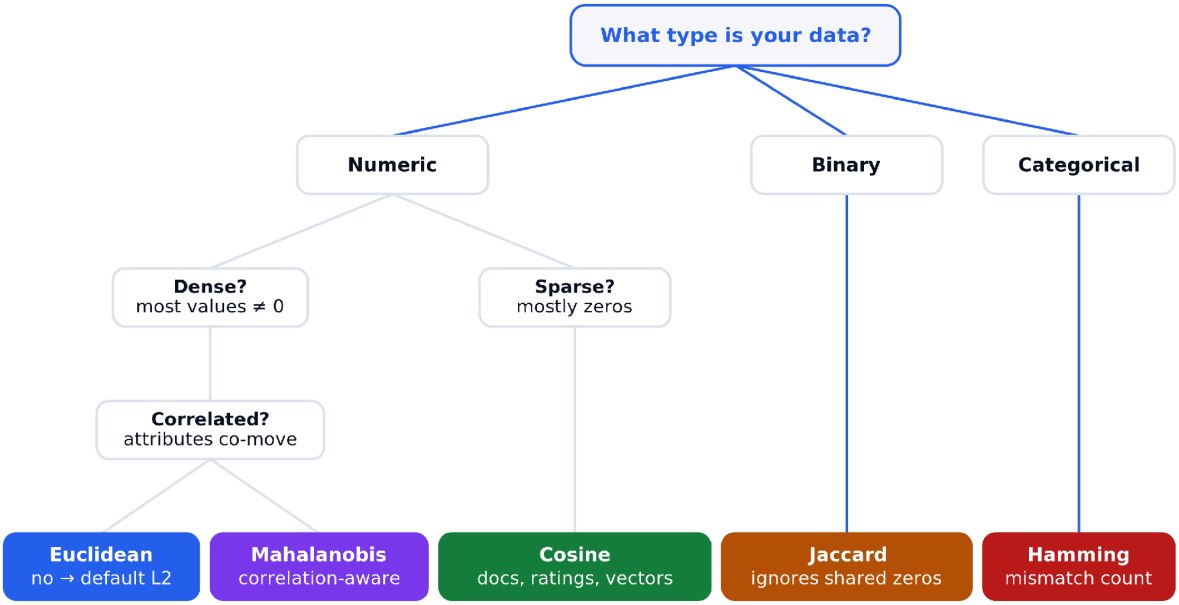
Proximity measures are not a niche mathematical curiosity - they are the computational core of several major machine learning tasks: clustering (grouping similar ShopCart customers into segments without labels), nearest-neighbour classification and recommendation, anomaly detection (flagging points that are dissimilar to everything else), and information retrieval (ranking documents by similarity to a query). Some methods, called kernel methods, go even further - they transform raw data straight into a similarity space and never look back at the original attributes.

3.3 A few facts that organize everything

- Jaccard and cosine similarity are built for sparse data - documents, user ratings, purchase baskets, where most attribute values are zero or absent.
- Euclidean distance and correlation are built for dense data - time series, multidimensional numeric data, where most attributes carry meaningful, non-zero values.
- Correlation captures only linear relationships; mutual information catches non-linear ones too.

3.4 Decision tree - which measure should I use?

The figure below distills the rest of Part 3 into a single flowchart. First ask what type the data is: numeric, binary, or categorical. For numeric data, ask whether it is dense or sparse; dense, uncorrelated data defaults to Euclidean distance, while dense data with correlated attributes calls for Mahalanobis distance, and sparse numeric data (ratings, embeddings, documents) calls for cosine similarity. Binary data uses Jaccard (for sparse, mostly-absent attributes); categorical data uses Hamming distance (a mismatch count). Every measure introduced in the rest of this Part sits somewhere on this tree.



Decision tree for choosing a proximity measure based on data type, density, and correlation structure.

3.5 Transformations between similarity and dissimilarity

Similarity and dissimilarity are two sides of the same coin, and it is often useful to convert between them - for example, most clustering algorithms expect a dissimilarity (distance), while some recommendation formulas are naturally expressed as a similarity.

$$d(x, y) = 1 - s(x, y) \qquad \text{when } s \in [0, 1]$$

A monotonically decreasing function of distance also works in place of the simple 1-minus-similarity form:

$$s = \frac{1}{1 + d} \qquad \text{or} \qquad s = e^{-d}$$

Both of these squash an unbounded distance into a bounded [0,1] similarity.

3.6 Proximity between single-attribute values

Before combining attributes into an overall proximity score, it helps to define proximity for a single attribute at a time, and the right definition depends on the attribute type.

Attribute type	Dissimilarity	Similarity
Nominal	d = 0 if same, 1 if different	s = 1 if same, 0 if different
Ordinal (mapped to 0..n-1)	d = |x-y| / (n-1)	s = 1 - d
Numeric (interval/ratio)	d = |x-y|	s = 1 / (1+d) (one option)

ShopCart example: comparing two customers on membership_tier alone (Bronze through Platinum) uses the ordinal row.

$$d = \frac{|x - y|}{n - 1} \qquad s = 1 - d$$

3.7 How equal are ordinal gaps, really?

A worked example exposes a subtlety in the ordinal row above. Product review quality is often coded as {poor, fair, OK, good, wonderful}, mapped to the integers {0, 1, 2, 3, 4}:

$$d(\text{good}, \text{OK}) = \frac{|3 - 2|}{4} = 0.25$$

This treats the gap between "fair" and "good" as exactly the same size as the gap between "OK" and "wonderful" - both differences of 2 map to a dissimilarity of 0.5. Whether that assumption is reasonable is a judgment call, not a mathematical fact: ordinal encodings impose equal spacing on categories that were never guaranteed to be equally spaced.

3.8 Minkowski Distance

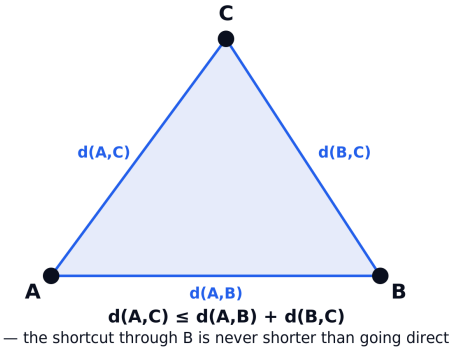
Minkowski distance is a single formula that generalizes several famous distances, parameterized by an order r:

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^{r} \right)^{\frac{1}{r}}$$

- r = 1: Manhattan distance (L1). For binary attributes, this is exactly the Hamming distance.
- r = 2: Euclidean distance (L2) - the familiar straight-line distance.
- r -> infinity: Chebyshev / supremum distance - the single largest per-attribute difference dominates entirely.

3.9 Distance as a metric

A distance function is called a metric when it satisfies three axioms. This matters because many algorithms - k-nearest-neighbours, k-means, and others - implicitly assume the proximity measure they are given behaves like a metric.



The triangle inequality, illustrated: the direct path from A to C is never longer than the detour through B.

$$d(x, y) \geq 0, \quad d(x, y) = 0 \text{ iff } x = y$$

Positivity: distance is never negative, and is zero exactly when the two points are identical.

$$d(x, y) = d(y, x)$$

Symmetry: the distance from x to y equals the distance from y to x.

$$d(x, z) \leq d(x, y) + d(y, z)$$

Triangle inequality: routing through a third point B is never a shortcut.

3.10 Is it actually a metric? Four worked checks

It is easy to assume any "distance-like" formula automatically satisfies the three axioms above. It does not. Four concrete checks make this precise.

Supremum distance (the r -> infinity case of Minkowski) is a genuine metric. For products x = (2,7,1) and y = (5,7,4) on three attributes:

$$d(x, y) = \max(|2-5|,\ |7-7|,\ |1-4|) = \max(3, 0, 3) = 3$$

Positivity, symmetry, and the triangle inequality all hold for the max operator in general, so supremum distance is a valid metric.

"Minimum difference across any attribute" (informally, the r -> negative infinity case) is not a metric. Product A = (Rs 10 price, 500 units in stock) and product B = (Rs 10 price, 5 units in stock):

$$d(A, B) = \min(|10-10|,\ |500-5|) = \min(0, 495) = 0$$

A and B are clearly different products, yet the minimum-difference rule reports a distance of zero, because they happen to agree on price. Positivity fails: two distinct objects have been assigned distance zero.

The set difference d(A, B) = |A - B| (using set-minus cardinality) is not a metric either. Let A = {a,b,c} and B = {a,b}:

$$A - B = \{a, b, c\} - \{a, b\} = \{c\} \ \Rightarrow\ d(A, B) = 1$$

$$B - A = \{a, b\} - \{a, b, c\} = \varnothing \ \Rightarrow\ d(B, A) = 0 \neq d(A, B)$$

Going from A to B "loses" one element (c), giving a distance of 1; going from B to A loses nothing, giving a distance of 0. Symmetry fails: the distance depends on the direction of comparison.

Symmetric difference, d(A, B) = |A XOR B| (elements in exactly one of the two sets), is a valid metric. Using the same sets plus C = {a}:

$$A \ominus B = \{c\}, \quad B \ominus C = \{b\}, \quad A \ominus C = \{b, c\}$$

$$d(A, C) = 2\ \leq\ d(A, B) + d(B, C) = 1 + 1 = 2 \quad \checkmark$$

The direct A-to-C gap never beats the detour through B - the triangle inequality holds, along with positivity and symmetry (since A XOR B always equals B XOR A by construction).

3.11 Similarities as functions

Similarity functions follow mirror-image rules. If s(x,y) is a similarity, symmetry, and positivity-like axiom typically holds, but the triangle inequality (or its analogue) usually does not.

$$s(x, y) = 1 \text{ iff } x = y$$

For similarities scaled to [0,1]: s(x,y) = 1 exactly when x = y.

$$s(x, y) = s(y, x)$$

Symmetry: s(x,y) = s(y,x).

3.12 Comparing two binary vectors

For n binary attributes, every pair of objects x and y reduces to four counts, which are used to build every binary coefficient in this section.

Symbol	Meaning
f00	attributes where x = 0 and y = 0
f01	attributes where x = 0 and y = 1
f10	attributes where x = 1 and y = 0
f11	attributes where x = 1 and y = 1

ShopCart example: x and y might be two customers' 500-category "did they buy from this category" vectors.

3.13 Simple Matching Coefficient (SMC)

The naive first guess is to count agreement on both the 1s and the 0s:

$$SMC = \frac{f_{11} + f_{00}}{f_{00} + f_{01} + f_{10} + f_{11}}$$

SMC performs poorly on sparse data, such as transactions or documents. Two customers who simply did not buy from 495 of 500 categories will look artificially "similar" purely because they agree on all those absences (a large f00 term) - the coefficient is dominated by the fact that both did nothing, not by what they actually bought.

3.14 Jaccard Coefficient

The Jaccard coefficient fixes SMC's sparsity problem by ignoring f00 entirely - it only counts categories that at least one of the two customers actually bought from:

$$J = \frac{f_{11}}{f_{01} + f_{10} + f_{11}}$$

This fixes SMC's blind spot for binary sparse data. For non-binary sparse data, such as word counts in documents, even Jaccard is not the right tool, which motivates the use of cosine similarity next.

> **INTERACTIVE CHECK-IN**
>
> ShopCart wants to compare two customers' 500-category purchase-basket vectors (1 = bought from this category, 0 = did not), with almost every entry being 0 for both customers. Jaccard, not SMC, is the right choice - the shared absences that dominate SMC carry no information about what the two customers actually have in common.

3.15 Cosine Similarity

Cosine similarity extends beyond binary vectors to any non-negative vector, and measures the angle between two vectors rather than the distance between their tips:

$$\cos(x, y) = \frac{x^T y}{\|x\| \, \|y\|}$$

- Cosine = 1 means an angle of 0 degrees: x and y point in the same direction and differ only in length.
- Cosine = 0 means an angle of 90 degrees: x and y share no common terms or attributes.
- Cosine depends only on the components that are non-zero in both vectors, which is exactly why it is well-suited to sparse data.

This is the same formula from Lecture 02, now reused to compare two ShopCart customers' category-spend vectors, or two product-review embeddings.

3.16 Where cosine similarity is used

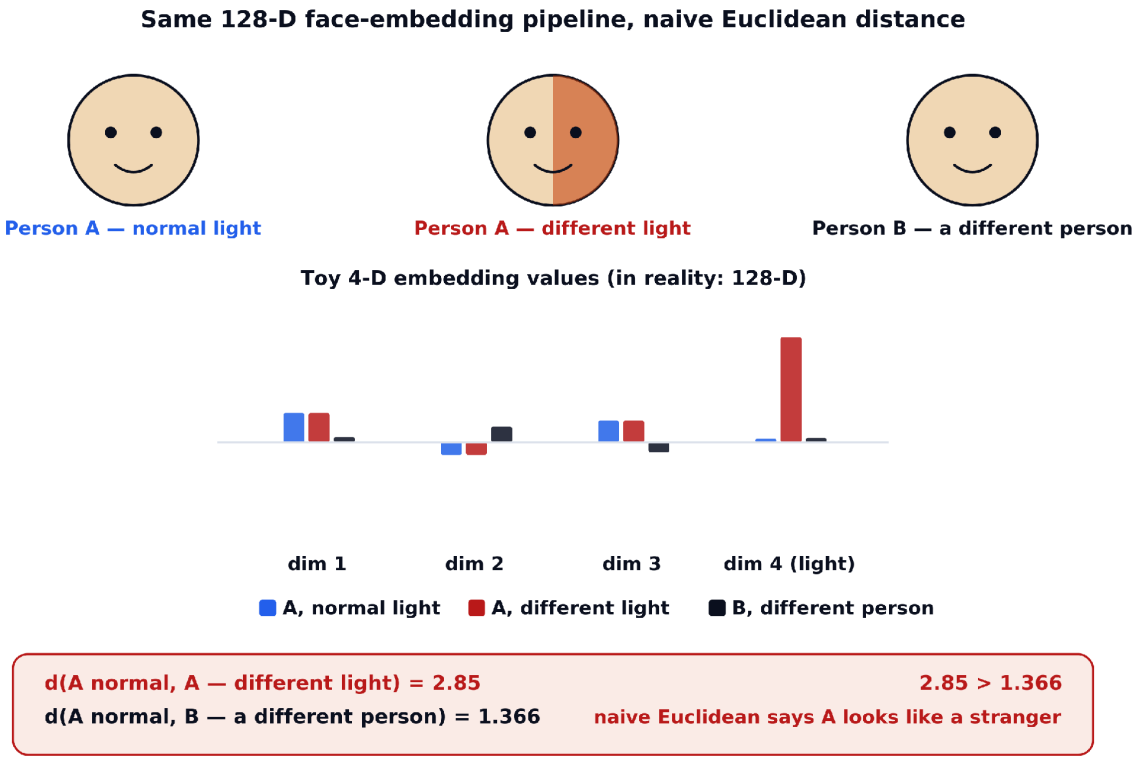
Cosine similarity is now the default distance measure of modern AI, because it is the only measure introduced so far that is built specifically for high-dimensional, sparse, direction-matters data - exactly the shape of a text embedding. It underlies ChatGPT-style embedding comparisons, sentence transformers (trained so that cosine similarity between outputs matches human judgements of meaning), retrieval-augmented generation (ranking which document chunks to hand an LLM before it answers), semantic search (matching "running shoes" to "jogging sneakers", which is impossible for

keyword search but easy for cosine on embeddings), and vector databases such as FAISS, Qdrant and Milvus, which are built almost entirely around computing this one formula at enormous scale.

3.17 Euclidean distance - and why it sometimes fails

Euclidean distance is the default distance almost everyone reaches for first, and under controlled, consistent conditions, it works well - for example, comparing two 128-dimensional face embeddings correctly separates "same person" from "different person" when lighting and pose are held fixed.

The failure mode appears when conditions change. Under a lighting or pose change, some embedding dimensions shift far more than others, and Euclidean distance treats every dimension as equally important - so it gets fooled. A concrete illustration with toy 4-dimensional embeddings (in reality 128-dimensional) makes this precise: Person A photographed under normal light and Person A photographed under different light should be recognized as the same person, and Person B is a genuinely different person.



A lighting change shifts one embedding dimension so much that, under naive Euclidean distance, Person A appears more dissimilar to themselves than to a different person, Person B.

Only one of the four toy dimensions is sensitive to lighting, yet because Euclidean distance sums squared differences across all dimensions equally, that single dimension dominates the total and produces exactly the wrong ranking: d(A normal, A different light) = 2.85, larger than d(A normal, B) = 1.366, even though A and B are different people.

SHOPCART PARALLEL

The same failure mode appears with ShopCart customers: Euclidean distance treats annual_income (range Rs 0 to Rs 50,00,000) and num_orders (range 0 to 50) as equally weighted attributes, which secretly means income dominates the entire distance calculation.

3.18 Mahalanobis Distance - the motivation

- Euclidean distance silently assumes every attribute is independent and equally scaled - almost never true in real data.
- ShopCart's annual_income and avg_order_value move together: high earners tend to place bigger orders.
- A customer who breaks that trend (low income, huge order) is genuinely unusual - but plain Euclidean distance cannot tell that apart from ordinary variation along the trend.

$$D_M(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

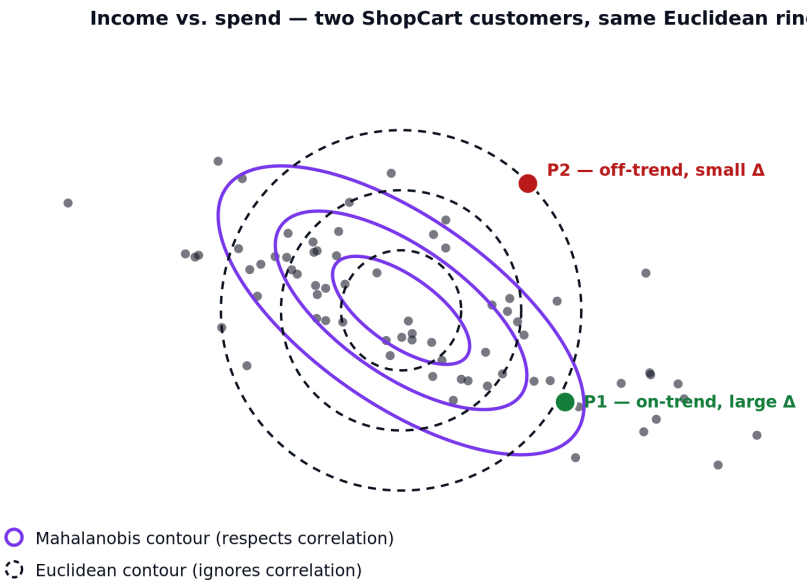
$$\Sigma = I \Rightarrow D_M(x, y) = d_{\text{Euclidean}}(x, y)$$

Setting the covariance matrix to the identity collapses Mahalanobis distance exactly back to Euclidean distance - Mahalanobis is a strict generalization, not a different idea.

The inverse covariance matrix, sigma inverse, is what lets the distance "see" correlation between attributes, exactly as it did back in Lectures 02-03.

3.19 Mahalanobis vs. Euclidean, visually

Plotting income against spend for a population of ShopCart customers, Euclidean distance contours are circles - they treat every direction as equally far from a reference point. Mahalanobis contours are ellipses that stretch along the natural income/spend trend. As a result, a customer who breaks that trend registers as far more unusual under Mahalanobis distance than a customer who simply sits further along the trend, even though both customers might sit on the same Euclidean ring around the center.



Euclidean contours (circles) versus Mahalanobis contours (ellipses aligned with the income/spend trend). P2, which breaks the trend, is far more anomalous under the Mahalanobis distance than P1, despite both sitting on the same Euclidean ring.

3.20 Distances, in code

```
import numpy as np
from scipy.spatial import distance

customers = np.array([
    [2, 7, 1], [5, 7, 4], [3, 6, 2], [4, 8, 3], [1, 5, 2], [6, 9, 5],
])                                  # [income_lakh, num_orders, tenure_yr]
a, b = customers[0], customers[1]
VI = np.linalg.inv(np.cov(customers.T))

distance.euclidean(a, b)            # L2
distance.cityblock(a, b)            # L1 / Manhattan
distance.chebyshev(a, b)            # L-infinity / Supremum
distance.jaccard([1, 0, 1, 1], [1, 1, 0, 1])  # binary sparse
distance.mahalanobis(a, b, VI)      # covariance-aware
```

SECTION SUMMARY

Part 3 built a toolbox: Minkowski distances (and the metric axioms that let algorithms trust them), Simple Matching and Jaccard coefficients for binary data, cosine similarity for sparse non-negative data, and Mahalanobis distance for correlated, unequally scaled numeric data. The decision tree in Section 3.4 is the map for choosing among them.

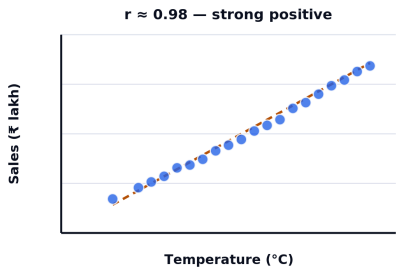
Part 4 - Correlation, Mutual Information, and Invariance to Transformation

Part 3 measured the closeness between two data objects. Part 4 turns to a related but different question: how strongly do two attributes move together? And once a proximity or relationship measure is chosen, does it continue to yield the same answer if the data are re-expressed in different units? Both questions turn out to matter enormously in practice, and the second one - invariance - is often the deciding factor in which measure is actually safe to use.

4.1 Correlation (Pearson)

Pearson correlation measures the strength and direction of a linear relationship between two paired sets of values:

$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\ \sqrt{\sum_i (y_i - \bar{y})^2}}$$

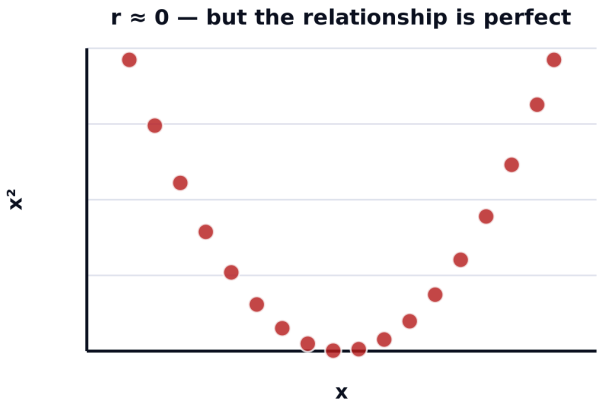


Ice-cream-maker sales are rising with daily temperature, a relationship close to a perfect positive linear correlation.

ShopCart's kitchen-appliance category shows the same pattern in real sales logs: hotter days, more ice-cream makers sold. A correlation of 1 (or -1) means a perfect positive (or negative) linear relationship, i.e., one variable is exactly an affine function of the other.

4.2 Correlation's blind spot

Correlation is completely blind to non-linear structure. The relationship y = x squared is perfectly deterministic - knowing x tells you y exactly - yet its Pearson correlation is approximately zero, because the relationship curves rather than trending consistently up or down.

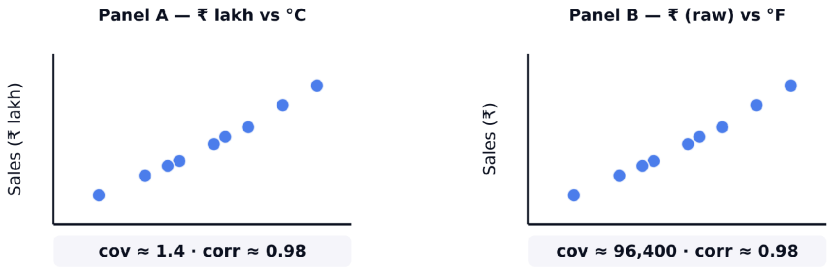


y = x squared: a perfect, fully deterministic relationship, yet Pearson correlation is close to zero because the relationship is not linear.

If correlation is 0, there is no linear relationship, but a non-linear one can still exist, and can even be perfectly deterministic. A measure that is not blind to curves is needed, which motivates mutual information later in this Part.

4.3 Correlation vs. Covariance

Covariance and correlation are two related but different numbers, and it is easy to conflate them.



Same relationship, same correlation — covariance's raw number swings wildly with the units you pick.

The same underlying relationship reported in two different unit systems: raw covariance changes drastically with units, while correlation does not.

$$\mathrm{Cov}(X,Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

Covariance signals only the direction of the linear relationship. Its magnitude depends on the units of x and y, so it cannot be compared across different attribute pairs.

$$r = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Correlation normalizes covariance by both standard deviations, producing a value in [-1, 1] that captures both strength and direction, and is comparable across any pair of attributes regardless of units.

4.4 Mutual Information (MI)

Mutual information measures how much information one set of values provides about another, whether the relationship is linear or not. Independent variables have MI equal to zero; completely dependent variables have MI at its maximum; MI has no fixed upper bound in general, but can be normalized into [0,1] for comparability.

4.5 Mutual Information, formally

MI is built from entropy - a measure of uncertainty.

$$MI(X,Y) = H(X) + H(Y) - H(X,Y)$$

$$H(X) = -\sum_{u} p(u)\log_2 p(u)$$

Entropy: the individual or joint uncertainty of a variable, computed from the probability of each value.

$$MI_{\mathrm{norm}} = \frac{MI(X,Y)}{\log_2\left(\min(m,n)\right)}$$

One common normalization divides by the log (base 2) of the smaller of the two variables' number of distinct values, giving a result in [0,1].

4.6 MI example: resolving the y = x squared cliffhanger

Section 4.2 left a cliffhanger: y = x^2 is a perfect relationship, yet the Pearson correlation is approximately zero. Mutual information resolves it, and working through the arithmetic by hand shows exactly where the numbers come from rather than dropping them into the formula unexplained.

Let x range over the seven equally likely integers -3, -2, -1, 0, 1, 2, 3, each with probability 1/7, and let y = x squared.

x	-3	-2	-1	0	1	2	3
y = x squared	9	4	1	0	1	4	9

Because y is a deterministic function of x, the seven (x, y) pairs are also equally likely. Reading off the table, y itself lands on only four distinct values: 0 (hit once, by x = 0), and 1, 4, 9 (each hit twice, by a plus/minus pair of x-values). So p(y=0) = 1/7, and p(y=1), p(y=4), and p(y=9) = 2/7.

First, the entropy of x. Since all seven values are equally likely:

$$H(X) = \log_2 7 = 2.8074 \text{ bits}$$

Second, the entropy of y, using its four-value distribution (1/7, 2/7, 2/7, 2/7):

$$H(Y) = 1.9502 \text{ bits}$$

Third, the joint entropy of x and y. Because y is completely determined by x, knowing x leaves no remaining uncertainty about y - the joint uncertainty of the pair equals the uncertainty of x alone:

$$H(X, Y) = H(X) = 2.8074 \text{ bits}$$

Now the three pieces combine using the definition from Section 4.5:

$$MI = (2.8074 + 1.9502) - 2.8074 = 1.9502 \qquad \frac{1.9502}{\log_2(4)} = 0.9751$$

Correlation said r is approximately 0 - "no relationship". Normalized mutual information says 0.9751 - "these two variables are almost completely dependent". Mutual information saw what correlation could not, because it makes no assumption that the relationship is linear.

> **WHERE THIS RESURFACES**
>
> The same entropy-based idea reappears every time a decision tree picks a split, or a transformer decides what to attend to - both are, underneath, information-theoretic calculations built from the same entropy formula introduced here.

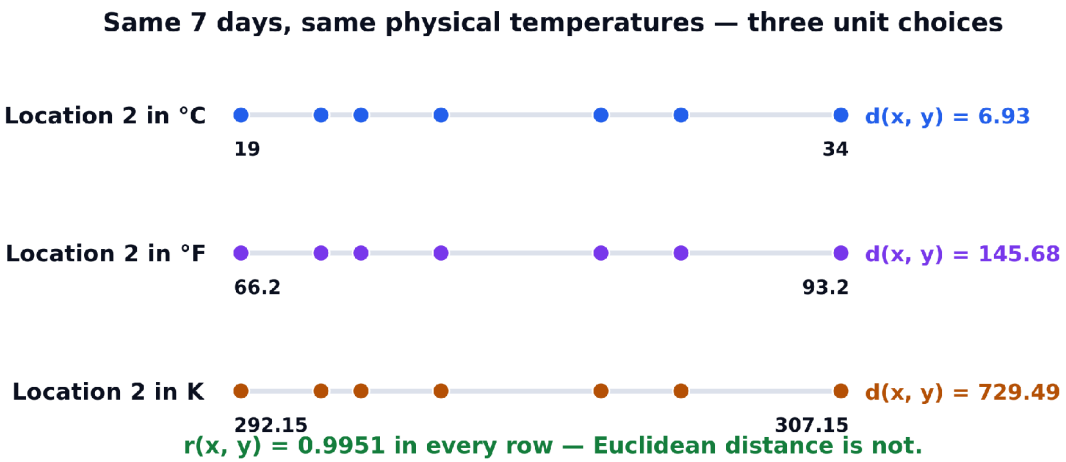
> **INTERACTIVE CHECK-IN**
>
> Two quick questions worth testing yourself on: is cosine similarity affected by the scale of a vector (multiplying every component by 10)? And which metric would a system like ChatGPT most likely use internally to find semantically similar sentences? (Answers: cosine similarity is unaffected by positive scaling of either vector, since it depends only on the angle between them - this is proved formally in Section 4.8; and cosine similarity over embedding vectors is exactly the tool of choice for semantic similarity, as discussed in Section 3.16.)

4.7 Invariance to Transformation

A proximity measure is invariant to a transformation if its value stays unchanged after that transformation is applied to the data. This sounds abstract, but it has an extremely concrete practical consequence: if a measure is not invariant to, say, a change of units, then the same underlying relationship can appear to have a completely different strength purely because of an arbitrary choice of measurement scale. The next three subsections work through this with real numbers.

4.8 Invariance example - Temperature

Seven days of temperature at one location, in Celsius, are compared against the same physical temperatures at a second location, measured in Celsius, Fahrenheit, and Kelvin. Fahrenheit both offsets and rescales the Celsius value ($F = C \times 1.8 + 32$); Kelvin only offsets it ($K = C + 273.15$). Which similarity measure gives the same answer regardless of which scale was used?



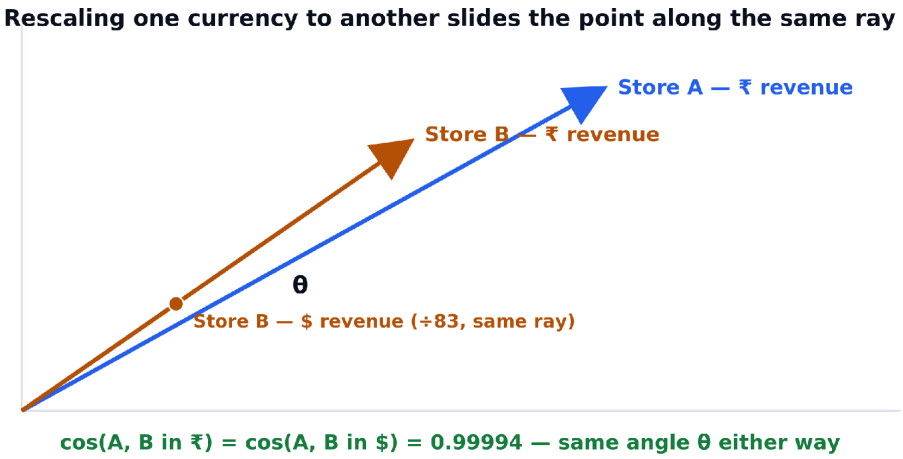
The same seven-day temperature pattern at Location 2, plotted on three different unit scales. Pearson correlation is identical across all three; Euclidean distance is not.

$$r(aY+b,\ x) = r(Y,x) \qquad \text{for any } a>0,\ b$$

Pearson correlation is unaffected by any positive linear rescale of either variable - and that is exactly what Fahrenheit and Kelvin are, relative to Celsius. Euclidean distance has no such protection: for the identical pair of days, it balloons from 6.93 (Celsius) to 145.68 (Fahrenheit) to 729.49 (Kelvin), purely because of the choice of units, with nothing about the underlying physical relationship having changed at all.

4.9 Invariance example - Currency

ShopCart reports daily revenue for two country stores: one in Indian rupees, one converted to US dollars - a pure scaling transform with no offset. Does cosine similarity between the two countries' daily revenue patterns change if one series is reported in rupees and the other in dollars?



Rescaling one store's revenue from rupees to dollars slides its vector along the same ray from the origin, leaving the angle to the other store's vector - and therefore the cosine similarity - unchanged.

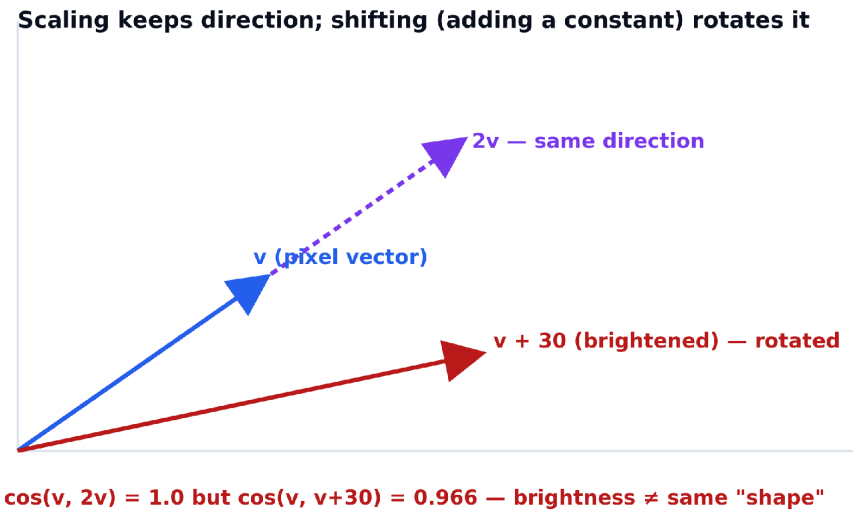
$$\cos(x,\ cy) = \cos(x, y) \qquad \text{for any } c > 0$$

No: cosine similarity is invariant to positive scaling, so the currency conversion (a pure multiplicative rescale) does not change the answer - it would change only if the transform included an offset, exactly as in the Fahrenheit case above. The same logic applies to height (centimeters vs. inches) or weight (kilograms vs. pounds): any pure unit rescaling leaves cosine and correlation unchanged, but not Euclidean distance.

4.10 Invariance example - Image Brightness and Document Length

A subtler trap involves shifting rather than scaling. Take a product photo x, and a version y with every pixel value shifted up by a constant (uniformly brighter). Cosine similarity between the raw pixel vectors is not invariant to this: an additive shift changes each vector's direction, not just its length.

Document length shows the mirror image. Let y be a review, y-doubled be the exact same review pasted twice (double length, identical term proportions), and y-different be a different, longer review with many new terms. Cosine correctly treats y and y-doubled as near-identical, because doubling every term count is a pure scaling operation; Euclidean distance does not, because it is sensitive to the raw magnitude of the vectors.



Scaling a vector (v to 2v) preserves its direction and therefore its cosine similarity to any reference vector; adding a constant (brightening) rotates the vector and changes cosine similarity.

> **THE PATTERN ACROSS ALL THREE EXAMPLES**
>
> Cosine and correlation survive scaling. Almost nothing but the Mahalanobis distance survives both scaling and correlation between attributes simultaneously. Raw Euclidean distance is invariant to neither offsets nor scale changes.

4.11 Different scales, and correlated attributes

- Attributes that are uncorrelated but on different ranges: standardizing (z-scoring) each attribute is usually enough to put them on a common footing before computing an ordinary Euclidean distance.
- Attributes that are both correlated and on different ranges: use Mahalanobis distance, which folds the inverse covariance matrix directly into the distance calculation, exactly as introduced in Section 3.18.

A concrete illustration, comparing a raw multi-channel distance against single-channel distances:

	Raw distance	Green channel only	Red channel only
Euclidean	14.7	4.12	—
Mahalanobis	6	2	—

PCA (principal component analysis) offers another route: transform the data into orthogonal principal components first, which removes correlation between attributes entirely before a distance is ever computed. This is covered in more depth in the dimensionality-reduction lectures later in the course.

4.12 Mixed attribute types, and unequal weights

Real ShopCart customer records mix quantitative attributes (income, number of orders) with qualitative ones (city, membership tier). A combined proximity measure needs to weight each attribute's individual dissimilarity and sum them, putting quantitative and qualitative contributions on a common scale. This construction is known as Gower's coefficient:

$$d(x, y) = \frac{\sum_{k=1}^{p} w_k \, \delta_k(x, y) \, d_k(x, y)}{\sum_{k=1}^{p} w_k \, \delta_k(x, y)}$$

$$d_k(x, y) = \frac{|x_k - y_k|}{\text{range}_k} \qquad \text{(quantitative attribute)}$$

$$d_k(x, y) = \mathbb{1}[x_k \neq y_k] \qquad \text{(categorical attribute)}$$

Here, d_k is the per-attribute dissimilarity (a normalized range gap for numeric attributes, a 0/1 mismatch indicator for categorical attributes), w_k is that attribute's weight, and delta_k equals zero only when the pair cannot be compared on attribute k at all - for instance, because of a missing value - and equals one otherwise.

A related, separate issue is that not all attributes should count equally toward similarity even when they are all comparable: a recommendation engine might reasonably want to weight recent_purchases far more heavily than signup_year. Weighted Minkowski distance and weighted cosine similarity handle this directly by attaching an explicit weight to each attribute's contribution.

4.13 Correlation and similarity, in code

```
import numpy as np
from scipy.stats import pearsonr
from sklearn.metrics import mutual_info_score
from sklearn. metrics.pairwise import cosine_similarity

temp = [22, 25, 28, 31, 20, 26, 33, 19, 24, 29]
ice_cream_sales = [140, 165, 190, 220, 110, 175, 240, 95, 155, 205]
x_binned = np.digitize(temp, bins=[23, 27, 31])
y_binned = np.digitize(ice_cream_sales, bins=[150, 190, 220])
a, b = [1, 0, 1, 1, 0], [1, 1, 0, 1, 0]

pearsonr(temp, ice_cream_sales)          # correlation
mutual_info_score(x_binned, y_binned)    # mutual information
cosine_similarity([a], [b])              # cosine similarity
```

SECTION SUMMARY

Correlation and mutual information both measure relationships rather than distance, but only mutual information catches non-linear structure. Invariance is the property that decides which measures are safe to compare across differently scaled or offset data: cosine and correlation are scale-invariant, Mahalanobis distance is scale-and correlation-aware, and raw Euclidean distance is invariant to neither.

Closing - One question, five lectures

All four roads explored across Lectures 02 through 06 lead back to the same question first opened in Lecture 02: how does a computer know what is similar, typical, or unusual? Every formula across these five lectures - mean and spread, distance and angle, correlation and mutual information, invariance - is machinery built to answer that one question, from a different angle each time.

Lecture 05-06 - the whole toolkit

```
import numpy as np
from scipy.spatial import distance
from scipy.stats import pearsonr
from sklearn. metrics.pairwise import cosine_similarity


customer_vec_1 = np.array([82000, 12, 3.5])   # income, orders, tenure_yr
customer_vec_2 = np.array([91000, 9, 2.0])
history = np.array([
    [82000, 12, 3.5], [91000, 9, 2.0], [65000, 15, 4.1],
    [77000, 11, 3.0], [58000, 20, 5.2], [98000, 7, 1.5],
])
cov_matrix = np.cov(history.T)
attribute_x = history[:, 0]
attribute_y = history[:, 1]
basket_a, basket_b = [1, 0, 1, 1, 0], [1, 1, 0, 1, 0]


distance.euclidean(customer_vec_1, customer_vec_2)
distance.mahalanobis(customer_vec_1, customer_vec_2, np.linalg.inv(cov_matrix))
cosine_similarity([customer_vec_1], [customer_vec_2])
pearsonr(attribute_x, attribute_y)
distance.jaccard(basket_a, basket_b)
```

Quick reference - Lecture 05-06

Concept	Python
Euclidean / Manhattan / Chebyshev	scipy.spatial.distance
Mahalanobis distance	distance.mahalanobis
Cosine similarity	sklearn.metrics.pairwise.cosine_similarity
Jaccard coefficient	scipy.spatial.distance.jaccard
Pearson correlation	scipy.stats.pearsonr
Mutual information	sklearn. metrics. mutual_info_score

What is next

Every mean, distance, and similarity score computed across Lectures 04 through 06 has described the past. The next lecture, Fundamentals of Predictive Analytics, starts using them to predict what a ShopCart customer does next - the same dataset, a new question.